03 - Browser Extension Requirements

Visor DOM-to-Agent Context Compiler
DOC-03 | 0.1 requirements pack | Generated 2026-06-14

This specification defines the Chrome Manifest V3 extension architecture, permissions, message contracts, UI surfaces, and browser limitations.

Field	Value
Owner	Rohan PX / Visor project
Audience	AI coding agent, reviewer, future contributors
Status	Draft baseline for MVP build
Decision rule	MVP is local-first, minimal-permission, Chrome Manifest V3, no backend unless explicitly enabled.

Document control

Item	Value
Project	Visor DOM-to-Agent Context Compiler
Document code	DOC-03
Generated on	2026-06-14
Primary goal	Turn raw webpage DOM into safe, structured, agent-ready context.
MVP boundary	Chrome extension, local-only compiler, JSON/Markdown export, privacy redaction, tests.

1. Extension platform requirement

The MVP targets Chrome Manifest V3. The extension must use an event-driven background service worker, a content script for DOM access, and a popup/options UI for user control. Any cross-browser work must come after the Chrome MVP is stable.

2. Manifest requirements

```
{
  "manifest_version": 3,
  "name": "Visor DOM Context Compiler",
  "version": "0.1.0",
  "description": "Compile the current webpage into safe agent-ready context.",
  "permissions": ["activeTab", "scripting", "storage"],
  "host_permissions": [],
  "background": { "service_worker": "service-worker.js", "type": "module" },
  "action": { "default_popup": "popup.html" },
  "options_page": "options.html"
}
```

3. Required extension modules

Module	File examples	Responsibilities
Popup	src/popup/*	Compile button, mode selector, preview, copy/export.
Service worker	src/background/service-worker.ts	Message router, active tab lookup, settings access, compile orchestration.
Content script	src/content/content-script.ts	DOM extraction and page snapshot generation.
Options page	src/options/*	Default mode, token budget, privacy level, selectors, debug mode.
Shared types	src/shared/types.ts	CompileRequest, PageSnapshot, AgentContext, errors.
Storage adapter	src/storage/*	Read/write extension settings and site profiles.

4. Permission policy

Permission	Reason	MVP policy
activeTab	Run on current tab after user gesture.	Required; preferred over broad host permissions.
scripting	Programmatically inject or coordinate content scripts.	Required for MV3 injection flows.
storage	Persist local settings and site profiles.	Required; store minimum data.
host_permissions	Allow automatic access to specific domains.	Avoid in MVP; add only when user creates site profile requiring it.
tabs	Read tab metadata beyond activeTab.	Avoid unless implementation proves activeTab is insufficient.

5. Message-passing contract

```
// popup -> service worker
{ type: 'VISOR_COMPILE_ACTIVE_TAB', payload: CompileRequest }

// service worker -> content script
{ type: 'VISOR_EXTRACT_DOM', payload: { requestId: string, settings: EffectiveSettings } }

// content script -> service worker
{ type: 'VISOR_DOM_SNAPSHOT', payload: PageSnapshot }

// service worker -> popup
{ type: 'VISOR_COMPILE_RESULT', payload: CompileResponse }
```

6. Error handling requirements

ID	Requirement	Priority	Acceptance signal
EXT-001	Show a clear error for restricted pages such as chrome://, file:// without permission, or web store pages.	P0	Popup displays restriction message.
EXT-002	Handle no-active-tab state.	P0	Popup shows no active page message.
EXT-003	Handle content script timeout.	P0	Timeout error includes retry and debug option.
EXT-004	Handle large DOM limit exceeded.	P0	Compiler uses partial snapshot with warning.
EXT-005	Handle storage read/write failure.	P1	Settings fallback to defaults and warning is logged.

7. Cross-browser future path

- Keep browser APIs behind a thin adapter so Firefox/Safari ports can be added later.
- Avoid Chrome-only APIs unless wrapped and documented.
- Use webextension-polyfill in V1 if cross-browser support becomes a goal.
- Do not compromise MVP speed by building multi-browser support too early.

Source and reference notes

These sources are used to ground platform/security assumptions. The implementation must still verify current platform behavior during development.

Reference	Why it matters
Chrome Extensions - Manifest V3 and service workers: https://developer.chrome.com/docs/extensions/	Grounding for MV3, background service worker model, permissions, and packaging.
MDN WebExtensions - Content scripts: https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Content_scripts	Grounding for content script behavior, host permissions, and page DOM access.
MDN WebExtensions - storage API: https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/storage	Grounding for extension storage concepts and permissions.